# TFRA-YOLO: Textile Feature Refinement and Reassembly YOLO for Closed-Loop Fabric Defect Inspection on Circular Knitting Machines

[TBD: Author Name(s)]

***Abstract*—Large circular knitting machines underpin high-throughput textile production, particularly for plain-knit fabric, one of the most extensively manufactured fabric types. Conventional inspection is commonly performed after production; therefore, a defect that emerges early can remain undiscovered while the machine continues knitting, causing a substantial length of defective fabric, material waste, and rework. This paper presents an in-process, closed-loop edge-cloud inspection workflow and TFRA-YOLO, a lightweight detector designed for this production setting. A proximity-switch rotation signal activates edge-side inspection; when a defect is detected, the edge device sends a local stop command and presents the candidate defect to an operator for secondary confirmation. The defect record and the local confirmation outcome are uploaded to the cloud for defect statistics and process-improvement analysis, completing a human-in-the-loop production loop. TFRA-YOLO starts from YOLOv8n and introduces a warp–weft texture encoding (WWTE) block that jointly models horizontal, vertical, and local texture responses, together with a local texture-guided feature reassembly (LTFR) block that replaces the two fixed top-down upsampling operations with local feature reassembly. On a real production-line dataset of 403 images covering horizontal defects, vertical defects, and holes, the retained test split contains 81 images and 91 instances. TFRA-YOLO obtains 66.3% AP and 96.9% AP50, improving YOLOv8n by 7.0 and 2.8 percentage points, respectively, with 3.39 M parameters and 8.9 G FLOPs at $640 \times 640$ input. These results support the usefulness of orthogonal texture representation and local pyramid reconstruction for lightweight, in-process plain-knit fabric inspection. The manuscript also states the deployment measurements still required before a production-performance claim is made.**

***Index Terms*—Fabric defect detection, circular knitting machine, edge intelligence, closed-loop inspection, feature pyramid, lightweight object detection.**

## I. Introduction

CIRCULAR knitting machines produce fabric continuously, so an overlooked defect can propagate before an operator can intervene. A practical inspection system must therefore address more than offline recognition accuracy: it must synchronize sensing with machine operation, make a local decision without waiting for a cloud round trip, preserve evidence for local human review, and convert confirmed cases into records for cloud-side defect statistics and process improvement. Earlier vision systems have demonstrated on-machine inspection for circular knitting processes [1], while recent work continues to identify rapid motion, illumination variation, and scarce abnormal samples as central challenges [2]. However, many learning-based defect-detection studies still evaluate an image classifier or detector in isolation from the production response loop.

[TBD: Affiliation, city, country, and corresponding author information.]

The target images in this work exhibit a second challenge. The fabric surface is dominated by regular knitted texture, whereas a horizontal defect, a vertical defect, or a hole can be thin, low contrast, and highly anisotropic. A compact detector must retain local texture detail at the high-resolution pyramid level while also exploiting context to distinguish a true defect from benign texture changes. General multi-scale detectors use feature pyramids [3] and bottom-up aggregation [4]; content-aware feature reassembly [5] further shows that interpolation need not use a spatially fixed kernel. At the same time, asymmetric convolutional paths can strengthen directional structure [6]. These ideas are relevant to knitted fabric, but must be adapted conservatively so that the edge model remains small.

Recent IoT-oriented detector studies further motivate a complete evaluation rather than an accuracy-only presentation. GMS-YOLO combines lightweight multi-scale extraction with edge-device evaluation in degraded visual conditions [7]; YOLO-Prism reports repeated-seed pyramid-refinement experiments and multi-platform latency [8]; and MEAR-EffiNet combines architectural ablation, qualitative inspection, and embedded deployment measurements [9]. Their presentation strategy informs the validation plan in this work, although neither their data nor their models are used here.

This paper makes the following contributions.

- It formulates a production-oriented, human-in-the-loop inspection workflow for a circular knitting machine. A rotation-derived signal gates edge inspection, a local defect decision requests a stop, an on-site operator confirms the case, and the detection-confirmation pair is uploaded for cloud-side defect statistics and process-improvement analysis.
- It proposes TFRA-YOLO, a YOLOv8n-based detector with a warp–weft texture encoding (WWTE) block and a local texture-guided feature reassembly (LTFR) block. WWTE preserves horizontal, vertical, and local texture cues; LTFR performs content-adaptive rather than fixed-neighbor top-down reconstruction.
- It reports a transparent test-set comparison against lightweight YOLO baselines and representative two-stage and transformer detectors, together with a component ablation. The retained result improves AP from 59.3% for YOLOv8n to 66.3% for TFRA-YOLO at a modest 0.38 M-parameter and 0.8 G-FLOP increase.

Unlike a fully autonomous control claim, the proposed workflow deliberately retains on-site operator confirmation. The cloud path is a record, statistics, and process-improvement mechanism; it is not part of the real-time control loop and is

not claimed to perform unattended online learning or safety-certified machine control.

## II. Closed-Loop Inspection Workflow

### A. Production Workflow

Fig. 1 illustrates the intended operating sequence. An operator starts the circular knitting machine. A proximity switch reports the machine rotation state to the edge side and enables image acquisition and inference. The edge unit analyzes captured fabric images locally. When its decision logic identifies a candidate defect, it sends a stop request through the machine-side control interface and records the associated image and prediction. The candidate is shown locally to the operator for secondary confirmation. After the operator handles the event and restarts the machine, the production cycle resumes. Detection records and local confirmation outcomes are uploaded to the cloud, where they are aggregated for defect-category statistics and process-improvement analysis.

Fig. 2 provides the actual on-machine installation. The wide view shows the circular knitting machine on the production floor. The upper inset highlights the camera directed at the knitting region, and the lower inset shows the edge-side touch display during inspection. Together with the workflow in Fig. 1, this installation evidence distinguishes the work from an offline-only image experiment.

The design follows the edge-intelligence principle that time-sensitive perception and action remain near the physical process, while selected evidence can be retained centrally. Related IoT work combines local detection with a cloud or digital representation for broader inspection data management [10]. Here, the control consequence is more direct: the event is escalated to a worker, not accepted as a final autonomous verdict. This provides a pragmatic route for handling rare defects while retaining the records needed to identify recurring defect patterns and process anomalies.

### B. Operational Boundaries

The workflow separates three decisions: (i) whether machine rotation enables inspection, (ii) whether the detector produces a candidate defect, and (iii) whether an operator confirms and disposes of the event. The reported recognition experiments validate only the second decision on a held-out image split. Actual acquisition-to-stop latency, machine stop distance, false-stop rate on defect-free fabric, availability of the control interface, and the operator-confirmation rate require site measurements and are therefore not fabricated in this draft. These measurements are specified in the accompanying material checklist.

## III. TFRA-YOLO

### A. Overview

TFRA-YOLO is built from the YOLOv8n detector [11]. As shown in Fig. 3, the backbone retains the original resolution hierarchy, but replaces the C2f output refinement at four stages with C2f–WWTE. In the head, the two top-down nearest-neighbor upsampling operations are replaced with LTFR. The standard concatenation, C2f fusion, bottom-up path, and three-scale detection head are retained. The detection scales are $P_3$, $P_4$, and $P_5$, corresponding to $80 \times 80$, $40 \times 40$, and $20 \times 20$ feature maps for a $640 \times 640$ input.

### B. Warp–Weft Texture Encoding

The term "warp–weft" is retained as the internal WWTE name because it denotes two orthogonal texture directions. In this circular-knit application, it does *not* imply that the fabric is woven; operationally, the branches are interpreted as horizontal and vertical texture encoders. For a C2f output $X \in \mathbb{R}^{C\times H\times W}$, WWTE applies depthwise horizontal, vertical, and local branches and then fuses them:

$$F(X) = \phi\left([\mathrm{DW}_{1\times 7}(X), \mathrm{DW}_{7\times 1}(X), \mathrm{DW}_{3\times 3}(X)]\right), \quad (1)$$

where $[\cdot]$ denotes channel concatenation and $\phi$ is a $1 \times 1$ convolution with normalization and activation. Because input and output channels match in the present implementation, the module uses residual fusion:

$$Y = X + F(X). \quad (2)$$

The long $1\times 7$ and $7\times 1$ depthwise kernels make the representation responsive to thin texture discontinuities in either image direction, while the $3\times 3$ path supplies local texture evidence. The design is related to, but not equivalent to, asymmetric convolution blocks [6]: it uses depthwise directional paths and a learned concatenation fusion within the existing C2f structure.

### C. Local Texture-Guided Feature Reassembly

Fixed nearest-neighbor upsampling copies a source value to each new location and cannot adapt to nearby texture. LTFR uses the content-aware reassembly principle of CARAFE [5] at each of the two top-down transitions. Given input $X$, a $1\times 1$ compression produces $H$ channels, with $H = \max(C/4, 16)$. A $3 \times 3$ encoder produces $s^2k^2$ kernel logits, where the scale factor is $s = 2$ and $k = 5$. Pixel shuffle followed by softmax yields a location-specific $5 \times 5$ kernel $K_{i,j}$:

$$K_{i,j} = \mathrm{softmax}\left(\mathrm{PS}\left(E_{3\times 3}(E_{1\times 1}(X))\right)_{i,j}\right). \quad (3)$$

For each output position, a $5\times 5$ neighborhood $\mathcal{N}_{i,j}(X)$ is unfolded from $X$, repeated to the enlarged grid, and reassembled as

$$Y_{i,j} = \sum_{u=1}^{25} K_{i,j,u}\mathcal{N}_{i,j,u}(X). \quad (4)$$

Thus, the reassembly weights adapt to local texture rather than being spatially shared. Fig. 4 details both modules.

## IV. Experimental Protocol

### A. Dataset

The data were collected from a real textile production line. The three object categories are *horizontal defect*, *vertical defect*, and *hole*. Table I summarizes the provided image-level split and instance counts. The test set contains 81 images with 91 labeled defects. This small, imbalanced test set makes

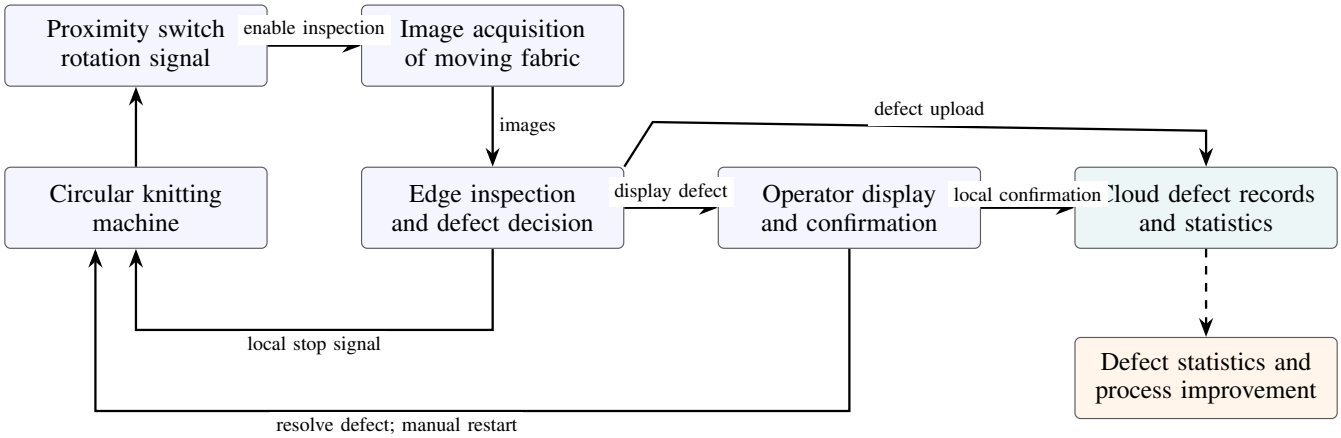



Fig. 1. Closed-loop inspection workflow for the circular knitting machine. Solid arrows describe the implemented production sequence provided for this study. The dashed cloud-side arrow represents defect statistics and process-improvement analysis; it is not part of the real-time control loop. Hardware-specific timing and interlock details are intentionally not inferred and should be added from site measurements before submission.

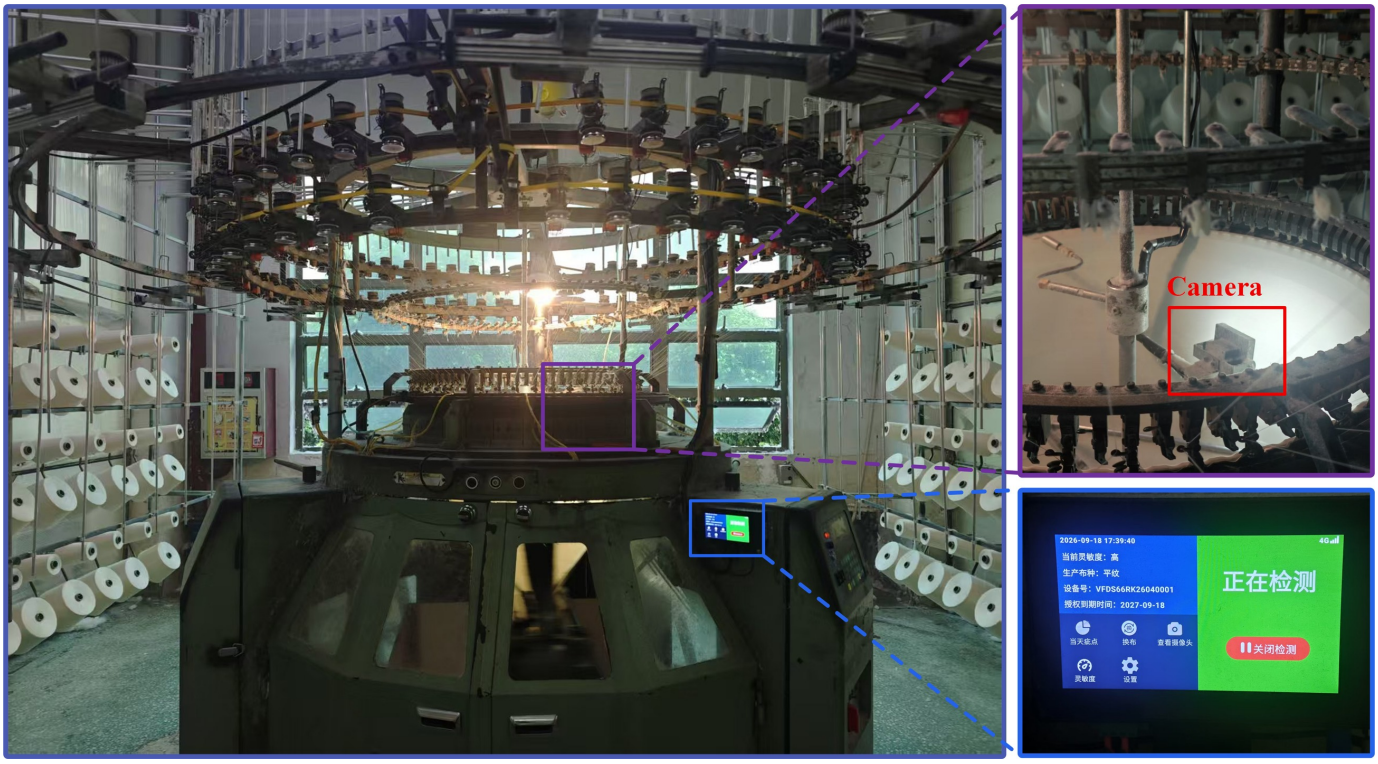



Fig. 2. Actual production-line installation on the circular knitting machine. The upper inset highlights the camera location; the lower inset shows the edge-side touch display while inspection is active. Colored callouts and insets are retained from the installation record.

every retained result useful for engineering comparison but is insufficient by itself to support a broad generalization claim. A future submission should state whether the split is grouped by production roll, machine, material, or time interval, and should add an independent production-period test set if available.

### B. Training and Evaluation

All models use the available pretrained initialization and $640 \times 640$ input. For the Ultralytics experiments, the retained configuration uses 300 maximum epochs, patience 80, batch size 16, automatic optimizer selection, and the framework's standard geometric and photometric augmentations; the TFRA-YOLO checkpoint is initialized from `yolov8n.pt`. Evaluation is conducted on the held-out test split with a maximum of 16 detections per image. We report COCO-style AP averaged over IoU thresholds 0.50:0.95 and AP50. AP

TABLE I
DATASET SPLIT SUPPLIED FOR THE CURRENT DRAFT.

| Split | Images | Instances | Horizontal | Vertical | Hole |
|---|---|---|---|---|---|
| Train | 282 | 329 | 174 | 75 | 80 |
| Validation | 40 | 48 | 24 | 13 | 11 |
| Test | 81 | 91 | 60 | 12 | 19 |

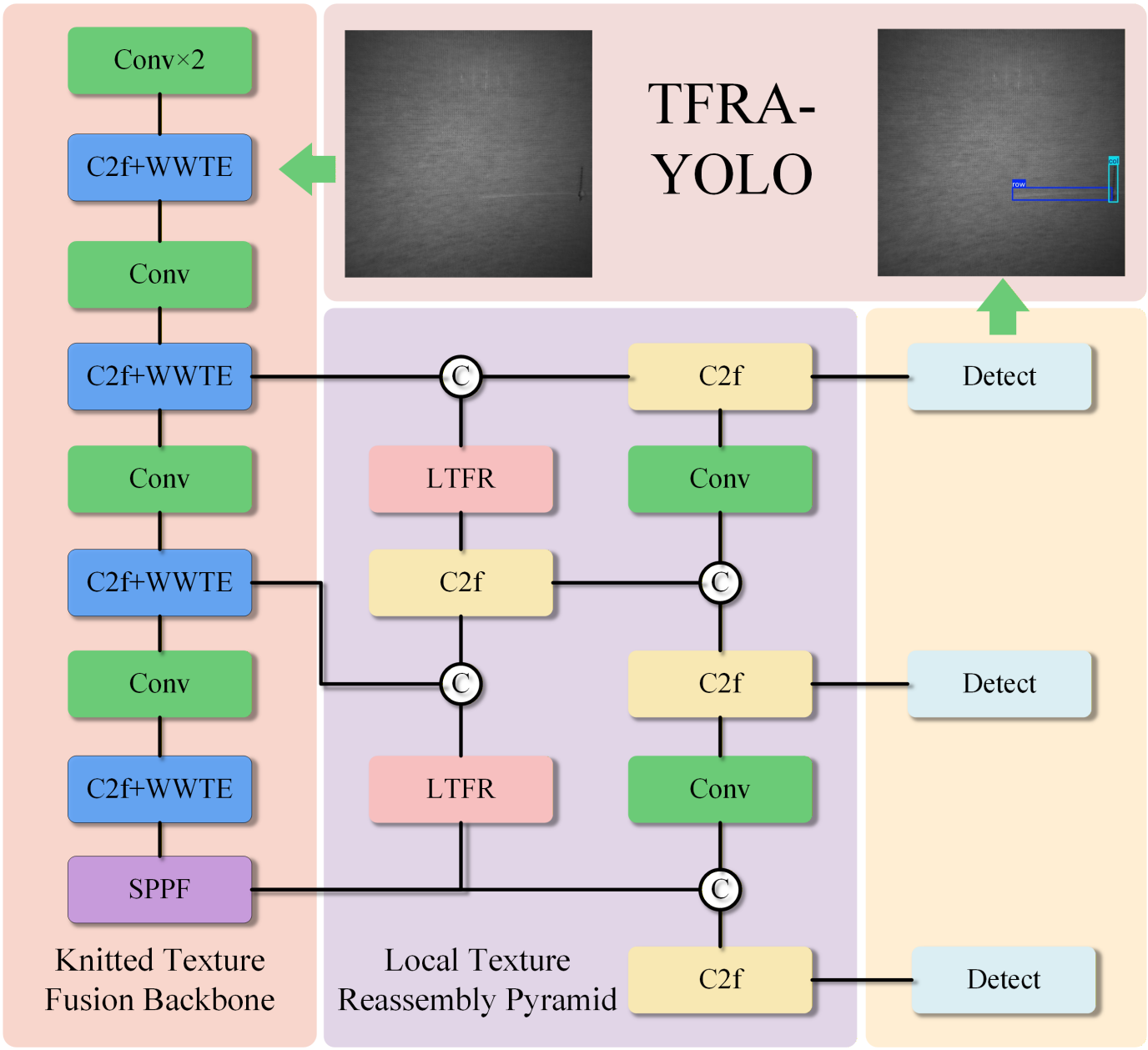



Fig. 3. Architecture of TFRA-YOLO. The Knitted Texture Fusion Backbone uses C2f–WWTE blocks to encode horizontal, vertical, and local texture cues. The Local Texture Reassembly Pyramid replaces the two top-down fixed upsampling operations with LTFR, while the bottom-up path and three-scale detection head follow YOLOv8n.

values are written as percentages.

The comparison table contains the latest retained checkpoint output for each model, rather than a mean over matched random seeds. FLOPs and parameter counts are reported at $[1, 3, 640, 640]$. The values are suitable for a coarse efficiency comparison, but cross-framework FLOP accounting can omit unsupported operators; no cross-framework latency claim is made. DETR, Deformable DETR, Faster R-CNN, and RT-DETRv2 follow their respective repositories and pretrained backbones [12]–[15].

## V. Results and Discussion

### A. Comparison With Detectors

Table II compares TFRA-YOLO with the retained baselines. TFRA-YOLO achieves 66.3% AP and 96.9% AP50. Relative to its direct YOLOv8n baseline, AP improves by 7.0 points and AP50 by 2.8 points for 0.38 M additional parameters and 0.8 G additional FLOPs. Among the compact YOLO variants evaluated here, TFRA-YOLO has the highest AP. RT-DETR-R18 obtains 67.7% AP, 1.4 points higher than TFRA-YOLO, but requires 20.09 M parameters and 60.72 G FLOPs. This is a useful trade-off rather than a claim that TFRA-YOLO dominates every detector.

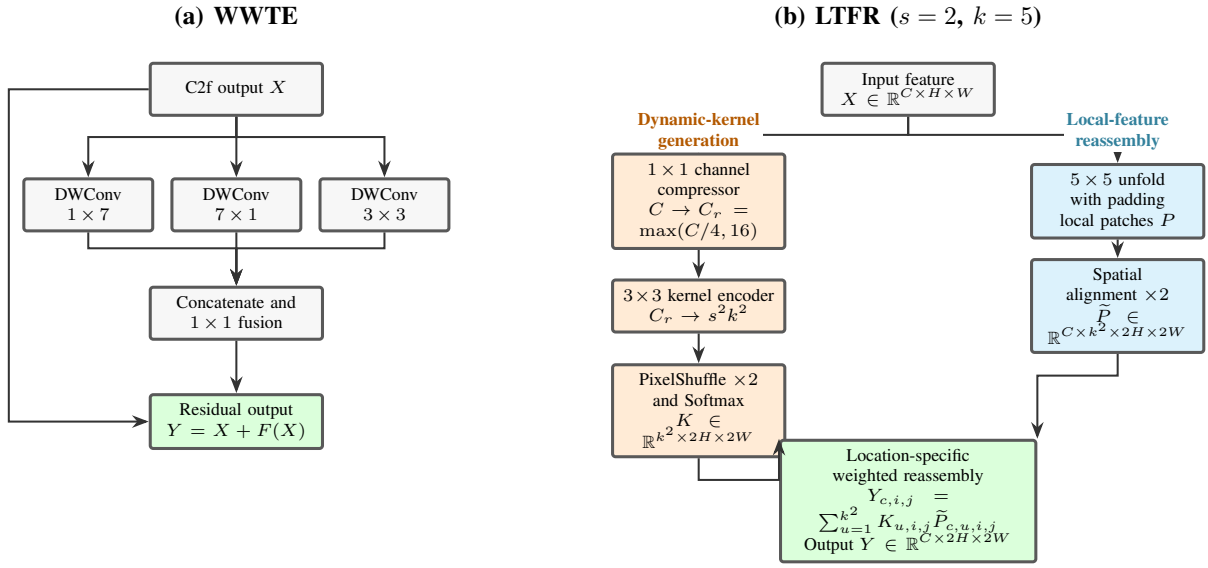



Fig. 4. Internal structures of (a) WWTE and (b) LTFR. WWTE uses three depthwise branches and residual fusion. LTFR is a compact CARAFE-style $2\times$ operator: it predicts location-specific $5 \times 5$ kernels and reassembles aligned local feature patches.

TABLE II
TEST-SET COMPARISON ON 81 IMAGES AND 91 INSTANCES. AP IS COCO AP@[.50:.95]. ALL VALUES ARE PERCENTAGES EXCEPT PARAMETERS AND FLOPS.

| Model | Params (M) | FLOPs (G) | AP50 (%) | AP (%) |
|---|---|---|---|---|
| YOLOv5n† | 2.50 | 7.10 | 93.1 | 58.4 |
| YOLOv8n | 3.01 | 8.10 | 94.1 | 59.3 |
| YOLO11n | 2.58 | 6.40 | 95.0 | 59.6 |
| YOLO12n | 2.56 | 7.30 | 94.3 | 60.9 |
| YOLO26n | 2.38 | 5.30 | 96.5 | 61.4 |
| RT-DETR-R18 | 20.09 | 60.72 | 95.2 | **67.7** |
| DETR-R50 | 41.28 | 77.25 | 96.4 | 53.6 |
| Deformable DETR-R50 | 39.82 | 158.66 | 96.8 | 64.3 |
| Faster R-CNN-R50 | 41.13 | 181.45 | 96.1 | 64.7 |
| TFRA-YOLO | 3.39 | 8.90 | **96.9** | 66.3 |

†The retained YOLOv5n checkpoint is the legacy YOLOv5n architecture, not the Ultralytics YOLOv5u variant.

Fig. 5 visualizes the parameter- and operation-accuracy trade-off without shifting any data coordinates. The position of TFRA-YOLO in the low-parameter region is consistent with the tabulated result. Because the horizontal axes are logarithmic, the visual is intended to show scale rather than imply a linear performance-efficiency relationship.

### B. Ablation

Table III isolates the two modifications with the same YOLOv8n family. WWTE contributes the larger retained isolated improvement (+4.8 AP), while LTFR contributes +1.6 AP. Combining both gives 66.3% AP, which is 0.6 points above the arithmetic sum of the isolated gains. This suggests that the directional representation and adaptive reassembly are complementary: WWTE supplies direction-specific texture cues, whereas LTFR transfers them through the top-down path. However, the four retained checkpoints are not matched-seed repetitions, so this interpretation is preliminary. Before submission, this table should be rerun with identical seeds, training budget, and reporting of mean and standard deviation.

TABLE III
ABLATION ON THE TEST SPLIT. THE CURRENT RETAINED RUNS USE THE SAME ARCHITECTURE FAMILY AND TRAINING RECIPE, BUT ARE NOT MATCHED-SEED REPETITIONS.

| Variant | WWTE | LTFR | Params | FLOPs | AP50 | AP |
|---|---|---|---|---|---|---|
| YOLOv8n | – | – | 3.01 | 8.1 | 94.1 | 59.3 |
| + WWTE | ✓ | – | 3.28 | 8.8 | 94.2 | 64.1 |
| + LTFR | – | ✓ | 3.11 | 8.2 | 94.3 | 60.9 |
| TFRA-YOLO | ✓ | ✓ | 3.39 | 8.9 | 96.9 | 66.3 |

### C. Per-Class and Qualitative Results

The TFRA-YOLO checkpoint obtains 61.7%, 73.8%, and 63.4% AP for horizontal defects, vertical defects, and holes, respectively. The notably high vertical-defect AP should be interpreted with care because the test category has only 12 instances. Fig. 6 presents seven selected images and the available standardized renderings. The visual comparison is illustrative rather than a replacement for the full quantitative test set. It is particularly useful for examining thin vertical

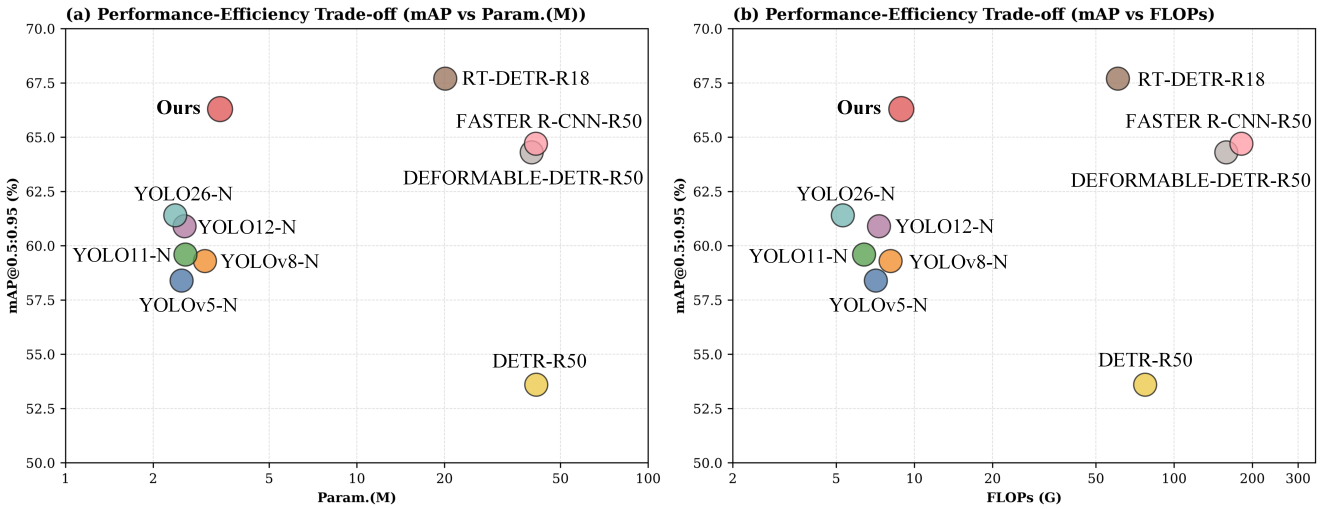



Fig. 5. Accuracy-efficiency trade-off of the retained test results. Both horizontal axes use a logarithmic scale. Marker coordinates are the exact values in Table II; the plot does not contain manual coordinate compression.

defects, long horizontal defects, and the hole class against repetitive background texture.

### *D. Limitations and Submission-Ready Validation*

The present evidence supports a promising lightweight detector on a real production-line image set, but it does not yet establish production reliability. First, the test split is small and category-imbalanced. Second, a direct comparison must use aligned training seeds and preferably report mean and standard deviation. Third, FLOPs are not a substitute for end-to-end edge latency, especially when image capture, preprocessing, postprocessing, control communication, and physical stopping are part of the operational loop. Finally, the operator's confirmation records are intended to improve defect statistics and process analysis; their actual acceptance rate and operational value have not yet been measured. These points define a concrete validation plan, rather than weaknesses that should be concealed.

## VI. Conclusion

This work presents a closed-loop framework for defect inspection on a circular knitting machine and TFRA-YOLO, a compact direction- and content-adaptive detector. The workflow couples rotation-aware edge inspection, local stop signaling, on-site human confirmation, and cloud-side defect statistics for process improvement. The detector combines WWTE for horizontal, vertical, and local knitted-texture encoding with LTFR for adaptive feature-pyramid reconstruction. On the retained test split, TFRA-YOLO improves YOLOv8n from 59.3% to 66.3% AP at 3.39 M parameters and 8.9 G FLOPs. The next step is not merely more offline accuracy: it is a disciplined site evaluation of sensing-to-stop latency, false-stop behavior, confirmation outcomes, process-analysis value, and grouped generalization across production conditions.

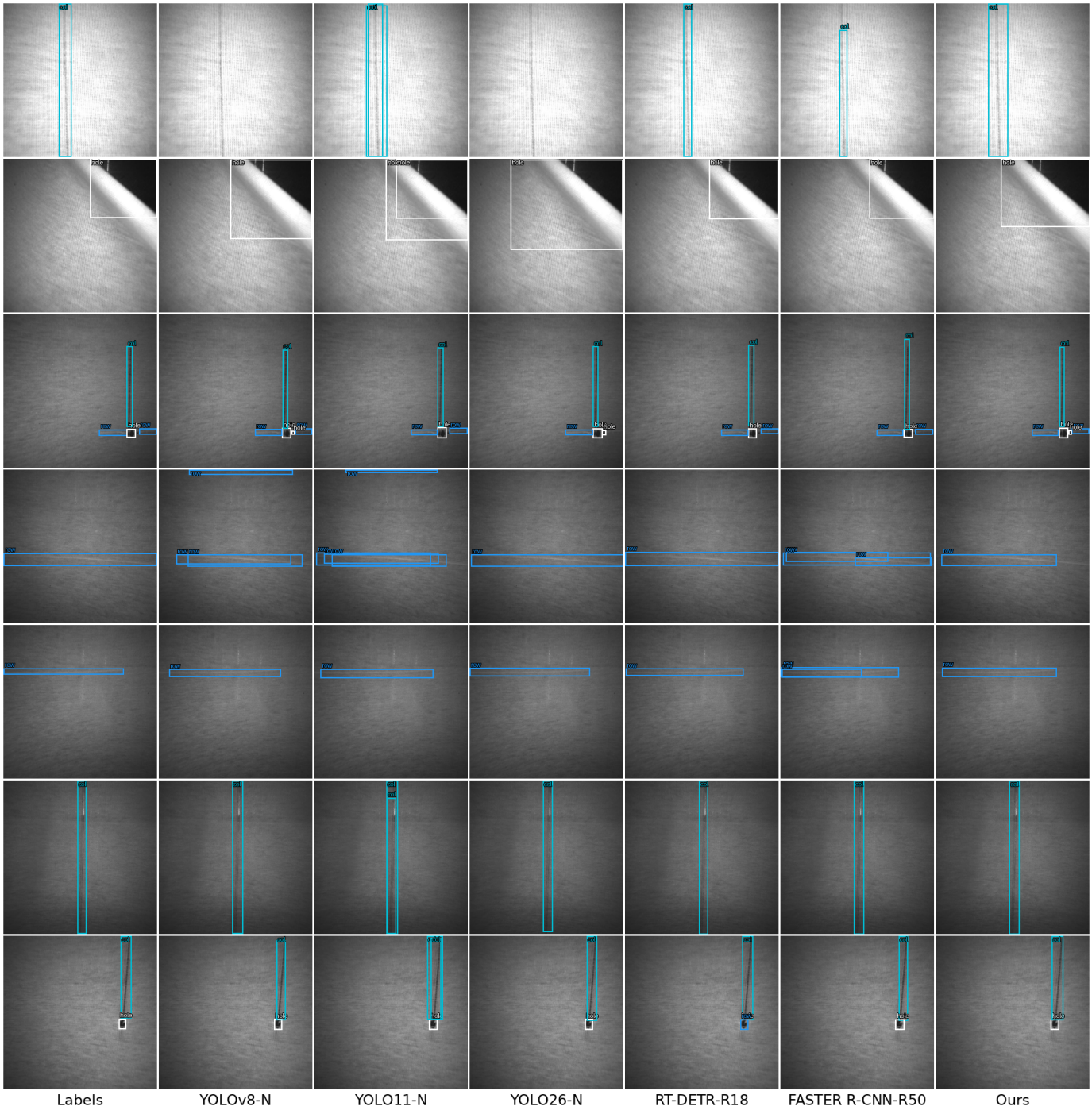



Fig. 6. Qualitative comparison on selected test images. Columns are labels, YOLOv8n, YOLO11n, YOLO26n, RT-DETR-R18, Faster R-CNN-R50, and TFRA-YOLO. The figure uses a common renderer for visual consistency; it should be updated after finalizing the exact sample-selection protocol and display confidence threshold.

## REFERENCES


[1] R. G. Saeidi, M. Latifi, S. S. Najar, and A. G. Saeidi, "Computer vision-aided fabric inspection system for on-circular knitting machine," *Textile Research Journal*, vol. 75, no. 6, pp. 492–502, 2005.

[2] Y.-Q. Ni, P.-K. Huang, C.-H. Yang, C.-C. Chang, W.-J. Wang, and D. Liang, "Real-time defect detection for fast-moving fabrics on circular knitting machine under various illumination conditions," *IEEE Access*, vol. 13, pp. 139 890–139 903, 2025.

[3] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 2117–2125.

[4] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path aggregation network for instance segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 8759–8768.

[5] J. Wang, K. Chen, R. Xu, Z. Liu, C. C. Loy, and D. Lin, "Carafe: Content-aware reassembly of features," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 3007–3016.

[6] X. Ding, Y. Guo, G. Ding, and J. Han, "Acnet: Strengthening the kernel skeletons for powerful cnn via asymmetric convolution blocks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 1911–1920.

[7] Y. Chen, Y. Wang, Z. Zou, and W. Dan, "Gms-yolo: A lightweight real-time object detection algorithm for pedestrians and vehicles under foggy

conditions," *IEEE Internet Things J.*, vol. 12, no. 13, pp. 23 879–23 890, 2025.

[8] H. Guo, W. Li, X. Yang, Y. Chen, and Y. Song, "Compute once, distribute many: Efficient feature pyramid refinement for ultra-lightweight detectors," *IEEE Internet Things J.*, 2026, early access.

[9] Z. Zeng, G. Zhao, H. Wu, C. He, and H. Xiao, "Underwater side-scan sonar object detection with multi-dimensional feature enhancement and adaptive attention routing network," *Ocean Engineering*, vol. 352, p. 124534, 2026.

[10] S. Saleh, A. Jolfaei, and M. Tariq, "Real-time pothole detection with edge intelligence and digital twin in internet of vehicles," *IEEE Internet Things J.*, vol. 12, no. 5, pp. 4852–4859, 2025.

[11] Ultralytics, "Ultralytics yolo," [Online]. Available: https://github.com/ultralytics/ultralytics, 2023, accessed: 2026-09-24.

[12] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 213–229.

[13] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable detr: Deformable transformers for end-to-end object detection," in *Proc. Int. Conf. Learn. Representations*, 2021.

[14] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *Adv. Neural Inf. Process. Syst.*, vol. 28, 2015.

[15] W. Lv, Y. Xu, Y. Yang, G. Li, J. Wang, L. Chen *et al.*, "Rt-detrv2: Improved baseline with bag-of-freebies for real-time detection transformer," *arXiv preprint arXiv:2407.17140*, 2024.